

Подробный отчет для защиты проекта

Итоговый проект — Python для разработки

ETL-пайплайн нормализации данных по доставкам
Витрины заказов и товаров (PySpark)

Docker Compose | PostgreSQL 15 | Airflow 2.8.1 | PySpark 3.5.0

1. Что мы делали и зачем

Что нам дали

Нам дали кучу данных о доставках из магазинов — 34 файла в формате Parquet (это как Excel, только для программистов — хранит данные в сжатом виде). Всего в этих файлах 9.7 миллионов строк и 25 колонок.

Проблема: данные ДЕНОРМАЛИЗОВАННЫЕ — всё свалено в одну огромную таблицу. Если человек заказал 5 товаров — его телефон записан 5 раз. Если курьера поменяли — все строки задваиваются.

Что нас попросили сделать

- Нормализовать данные — разбить одну таблицу на несколько связанных
- Загрузить в PostgreSQL через Apache Airflow
- Построить 2 витрины для аналитиков с помощью PySpark
- Упаковать в Docker, чтобы запускалось одной командой

Технологии — зачем каждая нужна

Технология	Зачем	Аналогия из жизни
Docker	Упаковывает приложение в контейнеры	Коробка IKEA — внутри всё для сборки
PostgreSQL	База данных для таблиц	Склад с полками — каждая таблица на полке
pgAdmin	Веб-интерфейс для просмотра БД	Как приложение банка для счёта
Airflow	Запускает ETL-задачи по порядку	Конвейер на заводе — этапы по порядку
PySpark	Считает витрины (метрики)	Калькулятор для миллионов строк
Python	Язык DAG-ов и скриптов	Основной язык проекта

Особенности данных

- 2 типа оплаты: «Сразу» (онлайн) или «Постоплата курьеру»
- Возможна смена курьера — в данных разные driver_id для одного заказа
- Возможна замена товара — есть и исходный товар, и товар-замена
- 3 причины отмены: «Покупатель отменил», «Ошибка приложения», «Проблемы с оплатой»
- 3 магазина: Орел, Москва, Тула
- 18 видов товаров в 6 категориях

2. Как запустить у себя на компьютере

Шаг 0: Установить Docker Desktop

Docker — это программа, которая создаёт «виртуальные компьютеры» (контейнеры) внутри вашего компьютера. Нам не нужно устанавливать PostgreSQL, Airflow, Java — Docker сделает это сам.

- Скачать: docker.com/products/docker-desktop
- Установить (далее-далее-далее)
- Запустить Docker Desktop, подождать зеленый значок кита
- Проверить: в терминале набрать `docker ps` — должна быть пустая таблица

ВАЖНО: Docker Desktop -> Settings -> Resources -> Memory -> поставить 4-6 GB. По умолчанию стоит 2 GB и контейнеры будут падать!

Шаг 1: Запустить проект

```
cd final_project
docker-compose up --build
```

Первый запуск: 5-10 минут (скачивание образов). Потом: 30 секунд. Не закрывайте терминал! Для фонового режима добавьте `-d`.

Шаг 2: Открыть Airflow

Сервис	URL	Логин	Пароль
Airflow (ETL)	http://localhost:8080	admin	admin
pgAdmin (БД)	http://localhost:5050	admin@admin.com	admin
PostgreSQL	localhost:5432	delivery_user	delivery_pass

Шаг 3: Запустить загрузку данных

- В Airflow: включить тумблер у `etl_load_normalized` -> нажать Trigger DAG
- Подождать ~12-15 минут (квадратики зеленеют один за одним)
- Когда все зеленые — включить `build_data_marts` -> Trigger DAG
- Подождать ~1 минуту — витрины готовы!

Шаг 4: Посмотреть данные в pgAdmin

- Открыть <http://localhost:5050>
- Add New Server: Name = delivery
- Connection: Host = postgres (НЕ localhost!), Port = 5432
- Database = delivery, Username = delivery_user, Password = delivery_pass
- Слева: Servers -> delivery -> Databases -> delivery -> Schemas -> delivery -> Tables
- ПКМ на таблицу -> View/Edit Data -> All Rows

Как остановить

```
docker-compose stop    # остановить (данные сохраняются)
docker-compose down -v # остановить и УДАЛИТЬ всё
```

3. Архитектура проекта

Файлы — что за что отвечает

```
final_project/
|
|-- docker-compose.yml  <-- Главный! Какие контейнеры запускать
|-- Dockerfile          <-- Как собрать образ Airflow+PySpark
|-- requirements.txt     <-- Python-библиотеки
|-- scripts/init_db.sql  <-- SQL создания таблиц (авто при старте)
|-- dags/etl_load_dag.py <-- DAG загрузки данных
|-- dags/mart_build_dag.py<-- DAG построения витрин (PySpark)
|-- data/*.parquet       <-- 34 файла с исходными данными
|-- normalization_report.pdf <-- Схема БД для преподавателя
|-- project_report.pdf   <-- Этот отчет
```

5 Docker-контейнеров

Контейнер	Что делает	Доступ
PostgreSQL (delivery)	Наша БД с таблицами и витринами	localhost:5432
PostgreSQL (airflow)	Служебная БД Airflow (логи, статусы)	внутренний
pgAdmin	Веб-интерфейс для просмотра данных	localhost:5050
Airflow Webserver	Веб-страница с кнопками запуска DAG	localhost:8080
Airflow Scheduler	Реально выполняет задачи + PySpark	внутренний

Зачем ДБА PostgreSQL? Airflow хранит свои служебные данные (логи, статусы) в отдельной БД. Мы не хотим мешать данные о доставках с внутренностями Airflow.

Что в Dockerfile

- Берём готовый образ Apache Airflow 2.8.1
- Добавляем Java 17 — нужна для работы PySpark (Spark написан на Java/Scala)
- Ставим PySpark 3.5.0 — фреймворк для вычислений над большими данными
- Ставим PostgreSQL JDBC драйвер — мост между Spark и PostgreSQL
- Ставим pyarrow + pandas — для чтения Parquet-файлов

4. Нормализация данных

Что такое нормализация — на примере

Представь Excel-таблицу:

Заказ	Покупатель	Телефон	Товар	Магазин	Адрес
001	Иванов	+7-999-111	Яблоки	Сберегайка	Ленина 10
001	Иванов	+7-999-111	Молоко	Сберегайка	Ленина 10
001	Иванов	+7-999-111	Хлеб	Сберегайка	Ленина 10
002	Петров	+7-999-222	Яблоки	Сберегайка	Ленина 10

Видишь проблему? «Иванов» и его телефон записаны 3 раза. «Сберегайка» — 4 раза. Если Иванов сменит телефон — надо обновить 3 строки. Забудешь одну — в базе будет два разных телефона. Это АНОМАЛИЯ ОБНОВЛЕНИЯ.

После нормализации делаем отдельные таблицы:

Покупатели:	1 Иванов +7-999-111
	2 Петров +7-999-222
Магазины:	1 Сберегайка Ленина 10
Товары:	1 Яблоки 100
	2 Молоко 80
	3 Хлеб 50
Заказы:	001 покупатель=1 магазин=1
	002 покупатель=2 магазин=1
Товары в заказе:	заказ=001 товар=1 кол-во=2
	заказ=001 товар=2 кол-во=1
	заказ=001 товар=3 кол-во=1

Теперь телефон Иванова хранится ОДИН РАЗ. Обновить — одна строка. Это и есть нормализация.

Нормальные формы простым языком

1НФ (первая)

Каждая ячейка = одно значение. Нельзя запихать список в одну ячейку. Наши данные уже такие.

2НФ (вторая)

Если ключ составной (order_id + item_id), то каждый атрибут зависит от ВСЕГО ключа. item_title зависит только от item_id -> выносим в таблицу items.

3НФ (третья) — мы выбрали эту

Нет транзитивных зависимостей. Что это?

order_id -> user_id -> user_phone

Телефон зависит от пользователя, а не от заказа -> выносим в таблицу users.

3НФ — стандарт для баз данных. Не слишком мало, не слишком много.

Результат: 7 таблиц

Таблица	Что хранит	Записей	Ключ (PK)	Почему отдельная
users	Покупатели	31 910	user_id	Телефон зависит от user, не от заказа
stores	Магазины	3	store_id	Адрес зависит от магазина
drivers	Курьеры	390	driver_id	Телефон зависит от курьера
items	Товары	18	item_id	Название зависит от товара
orders	Заказы	340 000	order_id	Центральная таблица (FK на users, stores)
order_items	Товары в заказе	4 212 103	(order, item)	Many-to-many: заказ <-> товары
order_drivers	Курьеры заказа	340 649	id (auto)	Для отслеживания СМЕН курьеров

Почему order_drivers отдельно?

Курьера могут сменить. Один заказ = 2 курьера: первого сняли, второй доставил. У снятого курьера delivered_at = NULL. Если хранить driver_id прямо в orders — потеряем информацию о смене.

Почему order_items отдельно?

В одном заказе 1-10+ товаров. Это связь «многие-ко-многим». Составной ключ (order_id, item_id) гарантирует уникальность.

6. ETL-процесс (Airflow DAG-и)

Что такое ETL

E = Extract (извлечь данные из Parquet)

T = Transform (разбить на таблицы, убрать дубликаты)

L = Load (загрузить в PostgreSQL)

Что такое DAG

DAG = Directed Acyclic Graph. Это набор задач со стрелочками «что после чего». Ациклический = нет заикливания. В Airflow DAG — это Python-файл с описанием задач.

DAG 1: etl_load_normalized (~12 мин)

Читает 34 Parquet-файла ПО ОДНОМУ (чтобы не упасть из-за нехватки памяти) и раскладывает данные по 7 таблицам.

Задача	Что делает	Время
truncate_tables	Очищает ВСЕ таблицы (TRUNCATE) для идемпотентности	1 сек
load_dictionaries	Собирает уникальных users/stores/drivers/items из 34 файлов	13 сек
load_orders	Загружает 340 000 заказов с дедупликацией по order_id	84 сек
load_order_items	Загружает 4 212 103 позиций заказов (самая долгая!)	~11 мин
load_order_drivers	Загружает 340 649 назначений курьеров	28 сек

Граф зависимостей

```
truncate -> dictionaries -> orders -> order_items (параллельно)
                                -> order_drivers (параллельно)
```

Что такое идемпотентность

Можно запустить DAG 1 раз или 100 раз — результат одинаковый. Каждый запуск СНАЧАЛА очищает таблицы, ПОТОМ загружает. Если упало на середине — перезапускаем и всё ок.

DAG 2: build_data_marts (~1 мин)

Использует PySpark для подсчёта витрин. Читает таблицы из PostgreSQL через JDBC, считает метрики, записывает обратно (mode=overwrite).

Почему PySpark а не SQL?

- По заданию: PySpark = 2 балла, SQL = 1 балл
- PySpark удобен для сложных оконных функций (ранжирование товаров)
- Показываем что умеем работать со Spark

Как работает Spark с PostgreSQL

- Spark запускается внутри контейнера Airflow (Java + PySpark установлены)
- Подключается к PostgreSQL через JDBC (Java-мост к БД)
- Читает таблицы в память Spark -> делает JOIN, GROUP BY, RANK()
- Записывает результат обратно в PostgreSQL

7. Витрина заказов (mart_orders)

Каждая строка = один день + один город + один магазин. Всего 1 101 строка.

Разрезы: year, month, day, city, store_id, store_address.

Колонка	Метрика	Как считается (простым языком)
turnover	Оборот	Кол-во * цена, применяем скидки товара и заказа
revenue	Выручка	Как оборот, но вычитаем отмененные единицы товара
profit	Прибыль	Выручка минус стоимость доставки
total_orders	Создано заказов	COUNT DISTINCT order_id
delivered_orders	Доставлено	Заказы где delivered_at заполнен
canceled_orders	Отменено	Заказы где canceled_at заполнен
canceled_after_del	Отмен после доставки	И canceled_at, и delivered_at заполнены
canceled_svc_err	Ошибки сервиса	Причина = Ошибка прил. / Проблемы с оплатой
unique_buyers	Покупателей	COUNT DISTINCT user_id
avg_check	Средний чек	revenue / total_orders
orders_per_buyer	Заказов/покупат.	total_orders / unique_buyers
revenue_per_buyer	Выручка/покупат.	revenue / unique_buyers
driver_changes	Смен курьеров	Заказы, где в order_drivers > 1 записи
active_drivers	Активных курьеров	COUNT DISTINCT driver_id

8. Витрина товаров (mart_items)

Каждая строка = день + город + магазин + товар. Всего 19 818 строк.

Разрезы: year, month, day, city, store_id, store_address, category, item_id, item_title.

Колонка	Метрика	Как считается
item_turnover	Оборот товара	qty * price * (1-item_disc%) * (1-order_disc%)
ordered_quantity	Заказано единиц	SUM item_quantity
canceled_quantity	Отменено единиц	SUM item_canceled_quantity
orders_with_item	Заказов с товаром	COUNT DISTINCT order_id
orders_with_cancel	Заказов с отменой	Заказы где canceled_qty > 0
is_most_popular_day	Популярный/день	RANK() по qty DESC = 1 -> true
is_least_popular_day	Непопулярный/день	RANK() по qty ASC = 1 -> true
is_most_popular_week	Популярный/неделя	Аналогично по неделям
is_least_popular_wk	Непопулярный/нед.	Аналогично по неделям
is_most_popular_mon	Популярный/месяц	Аналогично по месяцам
is_least_popular_mon	Непопулярный/мес.	Аналогично по месяцам

Популярность: RANK() OVER (PARTITION BY год,период,город,магазин ORDER BY qty DESC). Товар с макс. количеством за период = most_popular = true.

9. SQL-запросы для проверки

Запустите в pgAdmin после загрузки данных:

Проверить что всё загружено

```
SELECT 'users' as t, COUNT(*) FROM delivery.users
UNION ALL SELECT 'orders', COUNT(*) FROM delivery.orders
UNION ALL SELECT 'order_items', COUNT(*) FROM delivery.order_items
UNION ALL SELECT 'mart_orders', COUNT(*) FROM delivery.mart_orders
UNION ALL SELECT 'mart_items', COUNT(*) FROM delivery.mart_items;
```

Топ-5 дней по выручке

```
SELECT year, month, day, city, revenue, total_orders
FROM delivery.mart_orders
ORDER BY revenue DESC LIMIT 5;
```

Средний чек по городам

```
SELECT city, ROUND(AVG(avg_check), 2) as avg_check
FROM delivery.mart_orders
GROUP BY city;
```

Процент отмен по магазинам

```
SELECT store_address,
ROUND(SUM(canceled_orders)::numeric / SUM(total_orders) * 100, 1) as pct
FROM delivery.mart_orders
GROUP BY store_address;
```

Самый популярный товар за каждый месяц

```
SELECT year, month, city, item_title, ordered_quantity
FROM delivery.mart_items
WHERE is_most_popular_month = true
ORDER BY year, month, city;
```

10. Критерии оценки

Критерий	Баллы	Что мы сделали
docker-compose up --build работает	2/2	Проверено — собирается с нуля
Airflow, PostgreSQL, pgAdmin работают	2/2	5 контейнеров, auto user creation
DDL + схема + объяснение декомпозиции	2/2	PDF + init_db.sql + ER-диаграмма
DAG-и идиempotentны	1/1	TRUNCATE + overwrite каждый запуск
Витрины через PySpark (не SQL)	2/2	PySpark 3.5.0 через JDBC
Данные не искажены	1/1	100% записей загружено и проверено

ИТОГО: 10 / 10

11. Частые проблемы при запуске

Cannot connect to the Docker daemon

Причина: Docker Desktop не запущен.

Решение: Запустить Docker Desktop, подождать 30 сек.

Port is already allocated

Причина: На вашем ПК уже запущен PostgreSQL на порту 5432.

Решение: Остановить свой PostgreSQL или поменять порт в docker-compose.yml.

Task exited with return code -9

Причина: Не хватает RAM.

Решение: Docker Desktop -> Settings -> Resources -> Memory -> 4-6 GB.

Airflow не видит DAG-и

Причина: Airflow сканирует папку dags/ раз в 30 сек.

Решение: Подождать 1-2 минуты.

pgAdmin не подключается

Причина: В поле Host написали localhost.

Решение: Писать postgres — это имя контейнера в Docker-сети.

Было зеленое, стало красное

Решение: Кликнуть на красный квадратик -> View Log -> посмотреть ошибку. Обычно помогает перезапуск: Clear -> Confirm.